```
print("Parth Mahesh Madrewar")
print("Reg. No. : 241080036")
print("Experiment 5")
```

```
Parth Mahesh Madrewar
Reg. No. : 241080036
Experiment 5
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression



data = pd.read_csv("/content/electricity_cost_dataset.csv")
print(data.head())
```

```
   site area structure type  water consumption  recycling rate  \
0       1360      Mixed-use             2519.0              69
1       4272      Mixed-use             2324.0              50
2       3592      Mixed-use             2701.0              20
3        966    Residential             1000.0              13
4       4926    Residential             5990.0              23

   utilisation rate  air qality index  issue reolution time  resident count  \
0                52               188                     1              72
1                76               165                    65             261
2                94               198                    39             117
3                60                74                     3              35
4                65                32                    57             185

   electricity cost
0            1420.0
1            3298.0
2            3115.0
3            1575.0
4            4301.0
```

```
X = data.drop("electricity cost", axis=1)
y = data["electricity cost"]

X = pd.get_dummies(X, drop_first=True)
X = X.astype(float)
y = y.astype(float)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

class LinearRegressionScratch:

    def __init__(self, learning_rate=0.01, epochs=1000):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = 0
        self.loss = []

    def fit(self, X, y):

        rows, columns = X.shape


        # Start weights with zero
        self.weights = np.zeros(columns)

        for i in range(self.epochs):
            y_pred = np.dot(X, self.weights) + self.bias

            error = y_pred - y
            mse = np.mean(error ** 2)      #-> also known as lose function
            self.loss.append(mse)
```

```
            weight_gradient = np.dot(X.T, error) / rows
            bias_gradient = np.mean(error)
            self.weights = self.weights - self.learning_rate * weight_gradient
            self.bias = self.bias - self.learning_rate * bias_gradient

    def predict(self, X):
        return np.dot(X, self.weights) + self.bias



model = LinearRegressionScratch(learning_rate=0.01,epochs=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)


print("\nBias:")
print(model.bias)
print("\nWeights:")
print(model.weights)


#Performance
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("\nScratch Linear Regression")
print("MSE :", mse)
print("RMSE:", rmse)
print("R2  :", r2)


# Plot training loss
plt.plot(model.loss)
plt.title("Training Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE")
plt.grid()
plt.show()


# Linear Regression using Scikit-Learn
model2 = LinearRegression()
model2.fit(X_train, y_train)
y_pred2 = model2.predict(X_test)

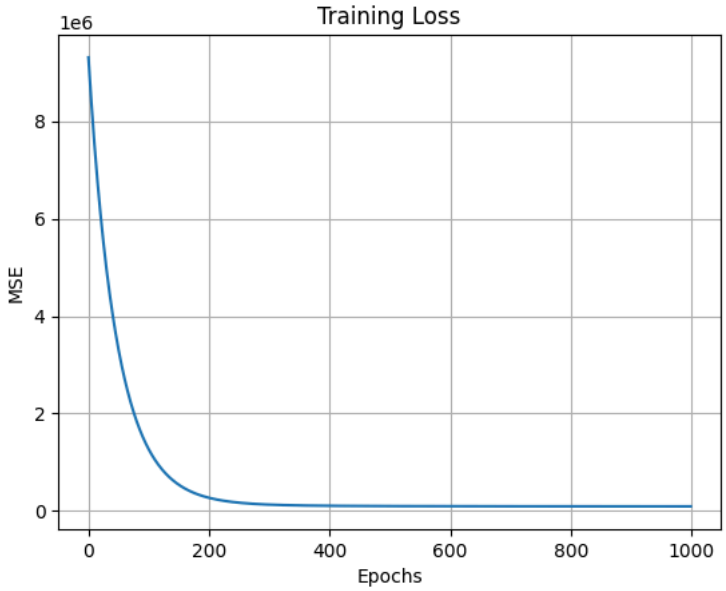

#Performance
mse2 = mean_squared_error(y_test, y_pred2)
rmse2 = np.sqrt(mse2)
r2_2 = r2_score(y_test, y_pred2)


print("\nScikit-Learn Linear Regression")
print("MSE :", mse2)
print("RMSE:", rmse2)
print("R2  :", r2_2)
```

```
Bias:
2844.3930737013966

Weights:
[ 772.27849265  134.27728615  -21.45656083  209.48135708   28.4346531
   42.08671748  254.86947712  185.72179331 -347.21337909  -78.28918132]

Scratch Linear Regression
MSE : 97845.10597397864
RMSE: 312.80202360914905
R2  : 0.921566613891106
```




```
Scikit-Learn Linear Regression
```